

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнила:

Ширкунова Мария

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Нужно привязать то, что мы делали в ЛР1 к внешнему API средствами fetch/axios/xhr. Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию, как в примерах, которые мы рассматривали в рамках тем "Имитация работы с API".

Сервис для управления личными финансами и бюджетом:

- Вход
- Регистрация
- Личный кабинет пользователя (счета, транзакции, бюджеты)
- Поиск и фильтрация транзакций по категории, сумме и дате
- Страница отчёта (графики расходов, категории, прогнозы)
- Интеграция с платежными аккаунтами (импорт транзакций, настройка правил)

Обязательно должен включать в себя следующие страницы: - Вход - Регистрация - Личный кабинет пользователя - Страница для поиска с возможностью фильтрации

Ход работы

В качестве основы был взят сверстанный в первой лабораторной работе интерфейс сервиса «Мои Финансы».

1. Настройка Mock-сервера и базы данных

Для имитации функциональности была установлена среда выполнения Node.js. С помощью пакетного менеджера npm установлены зависимости json-server и json-server-auth (в качестве среднего слоя для обеспечения безопасности).

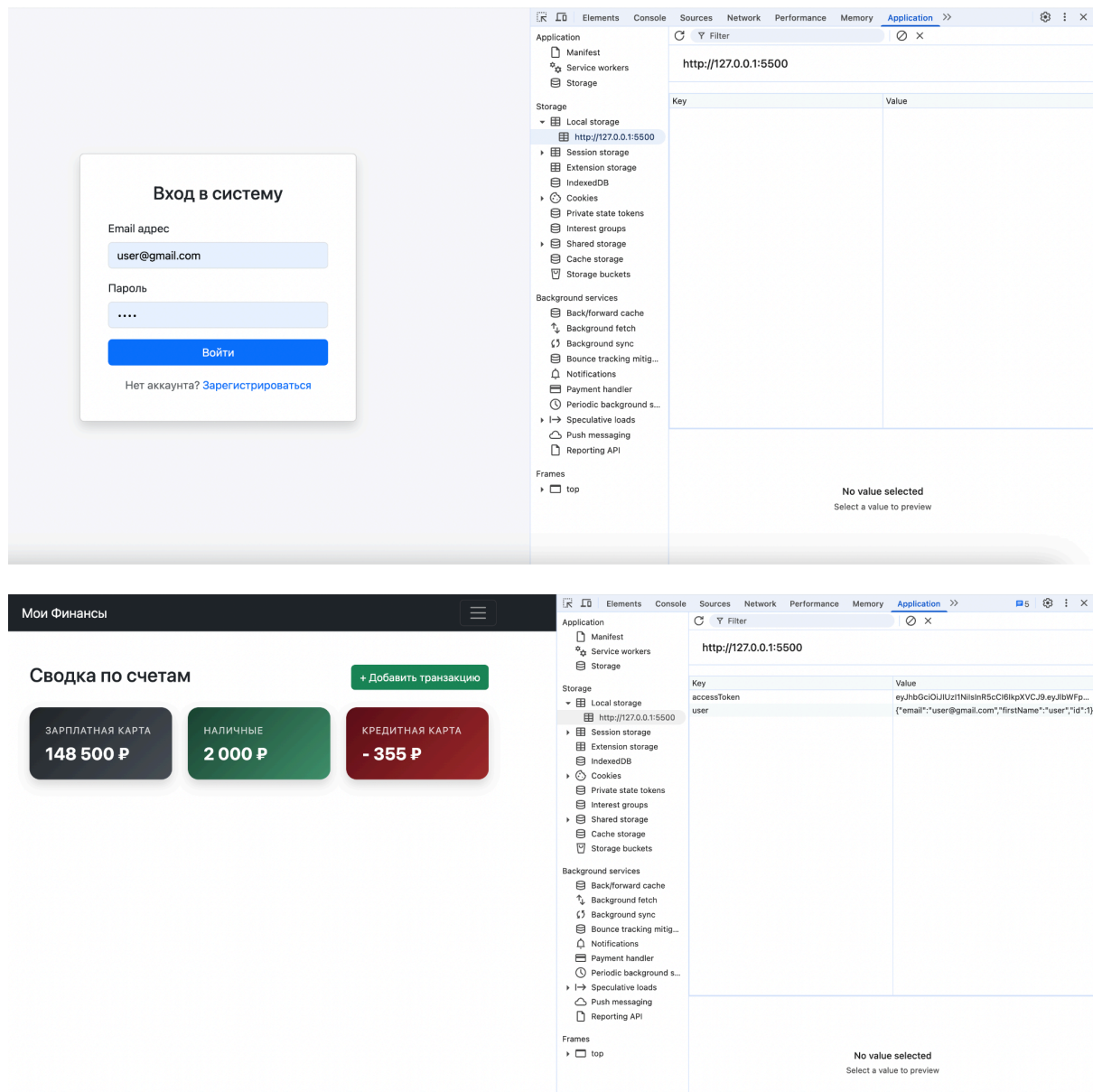
Был создан файл db.json, выступающий в роли базы данных. В нем сформированы две основные коллекции: users (для хранения зарегистрированных пользователей) и transactions (для хранения списка финансовых операций). Запуск сервера осуществляется на локальном порту (<http://localhost:3000>).

2. Реализация авторизации и регистрации

Логика входа и регистрации была полностью переписана. В HTML-формы добавлены атрибуты name для удобного сбора данных. Использован класс FormData, позволяющий извлечь все значения формы в один объект (`Object.fromEntries(formData.entries())`).

Отправка данных осуществляется асинхронно с помощью функции `fetch` (методом `POST`) на эндпоинты `/register` и `/login`. При успешном ответе от сервера полученный JWT-токен (`accessToken`) и информация о пользователе сохраняются в хранилище браузера (`localStorage`).

Рис. 1. Успешный `POST`-запрос авторизации и сохранение токена



Здесь показан результат работы JavaScript после отправки формы. Мой код сделал `POST`-запрос на сервер с логином и паролем. Сервер проверил пользователя в `db.json` и вернул ответ с уникальным токеном доступа (`accessToken`). Мой скрипт перехватил этот токен и с помощью команды `localStorage.setItem` сохранил его в память браузера (что и видно на скриншоте). Теперь этот токен будет прикрепляться к каждому

последующему запросу, чтобы сервер понимал, что я авторизована, и отдавал мне только мои транзакции.

3. Защита маршрутов и клиентская безопасность

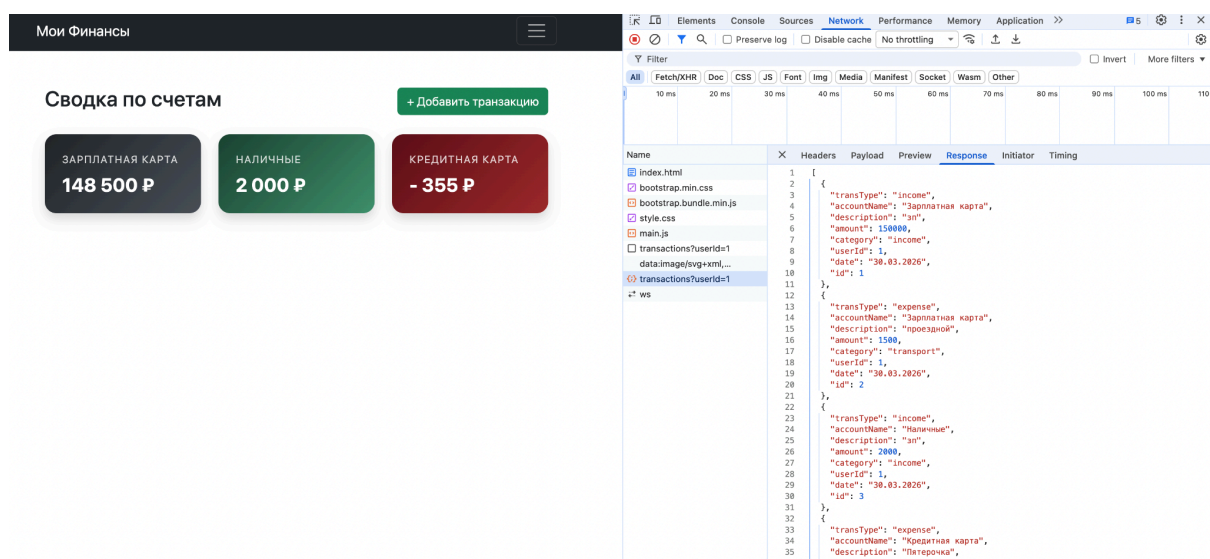
Написана функция `checkAuth()`, которая запускается при загрузке каждой страницы. Она проверяет наличие токена в `localStorage`. Если неавторизованный пользователь пытается зайти в личный кабинет, скрипт автоматически перенаправляет его на страницу входа (`login.html`). И наоборот: авторизованный пользователь не может попасть на страницу регистрации. Также обновлена кнопка «Выход», которая теперь не просто перекидывает на другую страницу, а вызывает функцию `logout()`, очищающую сессию в `localStorage`.

4. Динамический рендеринг и работа с приватными эндпоинтами

HTML-разметка личного кабинета была очищена от статических карточек счетов и строк таблицы транзакций — оставлены только пустые контейнеры-обертки (`#accountsContainer`, `#transactionsTable`).

Для запроса данных пользователя используется защищенный эндпоинт `/600/transactions`, доступ к которому возможен только при передаче заголовка `Authorization: Bearer <token>`. Использован параметр HTTP-запроса `?userId=...`, чтобы клиент получал только свои транзакции, а не операции других пользователей.

Рис. 2. Динамически выведенная сводка по счетам после GET-запроса

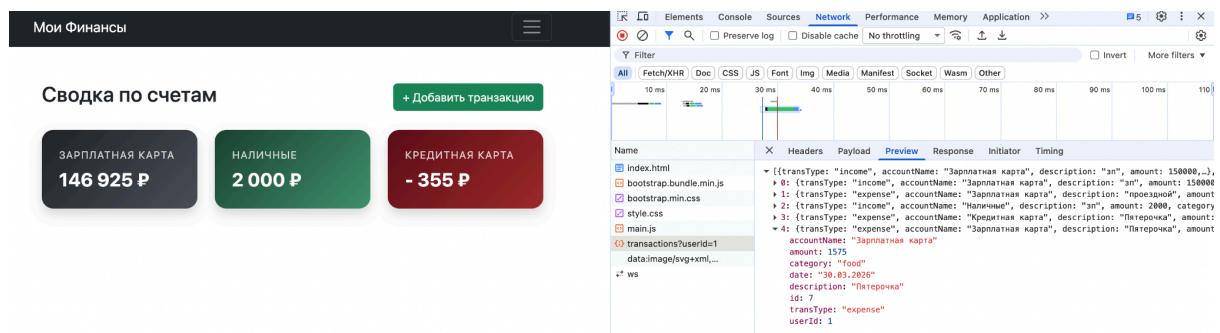


5. Расчет баланса и сохранение новых транзакций

Реализован продвинутый алгоритм вычисления баланса пользователя. Изначально баланс счетов равен нулю. При загрузке страницы функция `loadAccounts()` запрашивает историю `transactions` с сервера, перебирает массив (цикл `forEach`) и "на лету" вычисляет текущий баланс: прибавляя доходы и вычитая расходы в зависимости от типа операции. Затем карточки счетов генерируются с помощью шаблонных строк (`Template Literals`) и вставляются в DOM через `innerHTML`.

При заполнении формы «Новая транзакция» скрипт автоматически добавляет к объекту данных ID текущего пользователя и текущую дату. Выполняется POST-запрос. Если сервер отвечает успешно (код 200/201), модальное окно закрывается, форма очищается (`reset()`), а функции отрисовки списков вызываются повторно. Это позволяет обновить баланс и таблицу на экране моментально, без перезагрузки страницы (архитектура SPA).

Рис. 3. Процесс добавления новой транзакции (POST-запрос) и моментальное обновление баланса



6. Рефакторинг фильтрации

Алгоритм поиска по транзакциям был оптимизирован. Если в ЛР1 фильтрация работала путем чтения DOM-атрибутов у табличных строк, то теперь она работает с данными (Data-driven подход). Скачанные с сервера транзакции сохраняются в переменную, и метод фильтрации работает напрямую с массивом объектов JavaScript (`Array.prototype.filter()`), после чего таблица просто перерисовывается.

Вывод

В результате выполнения второй лабораторной работы статический макет интерфейса был успешно преобразован в функциональное веб-приложение, взаимодействующее с внешним API.

В ходе работы были изучены и применены на практике следующие технологии:

- JSON-server: Настроен локальный сервер-заглушка. Получено понимание того, как клиентская часть общается с сервером.
- Fetch API (асинхронные запросы): Выполнялись GET и POST запросы к серверу с помощью синтаксиса `async/await`. Изучена отправка и получение данных в удобном текстовом формате JSON.
- Авторизация пользователя: Настроена система регистрации и входа. Изучена работа с токенами: реализовано их сохранение в `localStorage`, чтобы браузер «запоминал» вошедшего пользователя и не просил вводить пароль при каждом обновлении страницы.
- JavaScript и работа с DOM: Отработан удобный способ сбора данных из форм с помощью `FormData`. Реализована отрисовка HTML-элементов через JavaScript (вывод таблицы транзакций из загруженного массива). Также настроен пересчет баланса счетов "на лету", благодаря чему интерфейс обновляется мгновенно, без полной перезагрузки страницы.